

**UNIVERSIDAD DEL VALLE
DE GUATEMALA**

Inteligencia Artificial 2026

Sección 10

Tercer Proyecto
Juegos Adversarios: Othello (Reversi)

June Herra

Dilary Cruz, 231010

Andres Mazariegos

GUATEMALA, mayo de 2026

Link de repositorio: <https://github.com/andresm220/Proyecto3IA>

Link de vídeo: <https://youtu.be/oCgYxB8oxXQ>

Índice

1. Introducción y objetivo	4
2. Diseño del sistema	4
2.1. Arquitectura	4
2.2. Clase <code>GameEngine</code>	5
2.3. Clase <code>GameVisualizer</code>	5
3. Algoritmos implementados	5
3.1. Minimax con poda Alpha-Beta (determinista)	5
3.2. Expectimax (probabilístico)	6
3.3. Monte Carlo Tree Search — UCT	7
3.4. Gestión del límite de tiempo	7
4. Heurísticas	8
4.1. Componentes heurísticos	8
4.2. Estrategia por fases	8
5. Análisis de rendimiento	9
5.1. Explosión combinatoria: Minimax puro vs. Alpha-Beta	9
5.2. Duelo de algoritmos: IA vs. IA	10
6. Conclusiones	12
7. Referencias	12

1. Introducción y objetivo

El objetivo de este proyecto fue diseñar e implementar un agente inteligente capaz de competir en **Othello (Reversi)**, un juego de suma cero y de información perfecta sobre un tablero de 8×8 . Se implementaron y compararon algoritmos de búsqueda **deterministas** (Minimax con poda Alpha-Beta) y **probabilísticos** (Expectimax y Monte Carlo Tree Search), evaluando su desempeño bajo una restricción estricta de **2 segundos por jugada**.

A diferencia de juegos triviales como Tic-Tac-Toe, en Othello el árbol de búsqueda es demasiado grande para explorarse hasta los estados terminales en la apertura y el medio juego. Por ello, el uso de **funciones heurísticas** para valorar estados no terminales fue obligatorio y determinante en la fuerza del agente.

2. Diseño del sistema

2.1. Arquitectura

El sistema mantiene una **separación total entre la lógica de decisión y la interfaz visual**, requisito del proyecto. El motor (**GameEngine**) y los agentes son módulos puros que no dependen de ninguna librería gráfica; la GUI (**GameVisualizer**, en Pygame) consume el motor pero nunca al revés. Esto permite ejecutar miles de partidas en el torneo IA-IA sin abrir ninguna ventana.

La estructura de archivos del repositorio es la siguiente:

```
Proyecto3IA/
+-- main.py                # Punto de entrada + menu de seleccion
+-- src/
|   +-- constants.py       # Constantes globales y matriz de pesos
|   +-- game_engine.py     # Motor de juego puro (sin GUI)
|   +-- heuristics.py      # 6 componentes heurísticos + fase
|   +-- agents/
|       +-- base_agent.py  # Clase base de agente
|       +-- alphabeta_agent.py # Alpha-Beta + iterative deepening
|       +-- mcts_agent.py   # MCTS / UCT
|       +-- expectimax_agent.py # Expectimax (oponente aleatorio)
|       \-- human_agent.py  # Agente humano (entrada por clic)
|   \-- visualizer.py      # GUI Pygame (GameVisualizer)
+-- analysis/
|   +-- combinatorial.py    # Analisis explosion combinatoria
|   +-- tournament.py       # Torneo 20 partidas IA vs IA
|   +-- plots.py            # Generador de graficas
|   +-- combinatorial.png   # Figura: nodos y b_eff
|   \-- tournament.png     # Figura: resultados del torneo
\-- tests/
    \-- test_rules.py       # Tests unitarios del motor de reglas
```

2.2. Clase GameEngine

Núcleo del juego. Gestiona el estado del tablero (matriz `numpy` de 8×8 con `EMPTY/BLACK/WHITE`), la generación de movimientos legales, la aplicación de jugadas con volteo de fichas en las 8 direcciones, el manejo automático del **pase** cuando un jugador no tiene jugadas válidas, la detección de estado terminal y el conteo final. Sus métodos principales son:

- `get_legal_moves(player)` — retorna todos los movimientos válidos para `player` (por defecto `current_player`).
- `apply_move(move)` — coloca la ficha, voltea las capturadas en las 8 direcciones y gestiona el pase automático si el rival no tiene jugadas.
- `is_terminal()` — detecta fin de juego (tablero lleno o ambos sin jugadas).
- `get_winner()` — retorna el ganador o 0 en caso de empate.
- `evaluate(player)` — delega en `heuristics.combined_evaluate()`.
- `clone()` — copia profunda barata del estado para uso en el árbol de búsqueda.

2.3. Clase GameVisualizer

Interfaz en Pygame que permite:

- Visualizar el tablero y el movimiento de las piezas en tiempo real, con animación de volteo de fichas.
- Mostrar indicadores de rendimiento de la IA por jugada: **nodos explorados**, **tiempo por jugada** y **valoración del tablero**.
- Seleccionar los modos **Humano vs. Humano**, **Humano vs. IA** e **IA vs. IA** desde el menú inicial.
- Elegir el algoritmo de IA: Alpha-Beta, MCTS o Expectimax.

3. Algoritmos implementados

3.1. Minimax con poda Alpha-Beta (determinista)

Minimax explora el árbol de juego asumiendo juego óptimo de ambos lados. La **poda Alpha-Beta** descarta ramas que no pueden influir en la decisión final, reduciendo el número de nodos visitados sin alterar el resultado.

Para respetar el límite de 2s se usó **profundización iterativa** (*iterative deepening*): se busca a profundidad 1, 2, 3, ... mientras quede tiempo, devolviendo siempre la mejor jugada de la última profundidad completada. Si el tiempo se agota durante una iteración, se lanza una excepción interna (`_Timeout`) que aborta esa iteración y retorna el mejor movimiento hallado hasta ese momento.

Se aplicó **ordenamiento de movimientos** para maximizar las podas: primero se prueban las esquinas (valor $-10\,000$ en la clave de orden), y el resto se ordena de mayor a menor peso posicional según la matriz `WEIGHTS` (Sección 6). Un buen orden acerca Alpha-Beta al límite teórico de $b^{d/2}$ nodos frente a b^d de Minimax puro.

Se reserva un margen de seguridad de **0.1 s** antes del límite de 2 s para garantizar que el agente siempre devuelva una jugada a tiempo.

Pseudocódigo simplificado

```
1 def get_move(engine):
2     deadline = now() + 2.0 - 0.1    # 0.1 s de margen de seguridad
3     best_move = legal_moves[0]
4     depth = 1
5     while now() < deadline:
6         try:
7             val, mv = search_root(engine, depth, deadline)
8             best_move = mv
9         except Timeout:
10            break                # devolver mejor de la iteracion anterior
11        depth += 1
12    return best_move
13
14 def alpha_beta(engine, depth, alpha, beta, maximizing, deadline):
15     if now() >= deadline: raise Timeout()
16     if terminal(engine): return terminal_value()
17     if depth == 0: return engine.evaluate(player)
18     sort_moves_by_priority(moves)    # esquinas primero
19     if maximizing:
20         val = -INF
21         for move in moves:
22             val = max(val, alpha_beta(child, d-1, alpha, beta, False, dl))
23             alpha = max(alpha, val)
24             if val >= beta: break    # poda beta
25         return val
26     else:
27         val = +INF
28         for move in moves:
29             val = min(val, alpha_beta(child, d-1, alpha, beta, True, dl))
30             beta = min(beta, val)
31             if val <= alpha: break  # poda alfa
32     return val
```

3.2. Expectimax (probabilístico)

Expectimax es una variante de Minimax que modela un oponente **subóptimo o incierto**. En lugar de asumir que el rival siempre minimiza, los nodos del oponente se tratan como **nodos de azar**: se promedian uniformemente los valores de todas sus jugadas posibles. Esto hace que el agente sea menos paranoico ante oponentes que no juegan de forma óptima.

En la implementación, los nodos del jugador propio son nodos *max* (igual que en Minimax), mientras que los nodos del oponente calculan la media aritmética de todos sus hijos. El agente también usa profundización iterativa y el mismo margen de 0.1 s.

Modelo de oponente: promedio uniforme sobre todas las jugadas legales del rival.

```

1 def expectimax(engine, depth, deadline):
2     if terminal(engine): return terminal_value()
3     if depth == 0:      return engine.evaluate(player)
4     moves = engine.get_legal_moves()
5     if engine.current_player == self.player: # nodo MAX
6         return max(expectimax(child, depth-1, dl) for child in children)
7     else: # nodo AZAR (promedio)
8         values = [expectimax(child, depth-1, dl) for child in children]
9         return sum(values) / len(values)

```

3.3. Monte Carlo Tree Search — UCT

MCTS construye el árbol de juego mediante cuatro fases iterativas:

1. **Selección** — desde la raíz, se desciende seleccionando el hijo con mayor valor UCT hasta alcanzar un nodo no completamente expandido o un nodo terminal.
2. **Expansión** — se añade un nuevo hijo al nodo seleccionado, eligiendo aleatoriamente uno de sus movimientos no explorados.
3. **Simulación (*rollout*)** — se juega una partida aleatoria uniforme hasta el estado terminal (o hasta un máximo de 200 jugadas de seguridad).
4. **Retropropagación** — el resultado (1,0 victoria, 0,5 empate, 0,0 derrota) se propaga hacia la raíz, actualizando visitas y victorias de cada nodo.

La selección utiliza la fórmula **UCT**:

$$\text{UCT}(n) = \frac{w_n}{v_n} + C \cdot \sqrt{\frac{\ln v_{\text{padre}}}{v_n}} \quad (1)$$

donde w_n es el número de victorias acumuladas en el nodo n , v_n el número de visitas, v_{padre} las visitas del padre, y C la constante de exploración. Se usó $C = \sqrt{2} \approx 1,41$, valor estándar en la literatura para UCT.

En lugar de un número fijo de iteraciones, el agente corre tantas simulaciones como permita el presupuesto de **2 s por jugada**. Al finalizar, se elige el hijo de la raíz con mayor número de visitas (criterio más robusto que mayor tasa de victoria).

Política de rollout: aleatoria uniforme sobre jugadas legales.

3.4. Gestión del límite de tiempo

Los tres agentes IA respetan el tope de 2.0 s: Alpha-Beta y Expectimax mediante profundización iterativa que aborta la iteración en curso al detectar `time.perf_counter() >= deadline`; MCTS mediante un bucle acotado por tiempo. En todos los casos se reserva un margen de seguridad de 0.1 s para garantizar que el retorno de la jugada no exceda el límite absoluto.

4. Heurísticas

Como en Othello no es viable llegar a estados terminales en la mayor parte de la partida, la función `evaluate()` combina **seis componentes**, todos normalizados al rango $[-100, 100]$ desde la perspectiva del jugador a mover.

4.1. Componentes heurísticos

1. **Paridad de fichas** (`coin_parity`): diferencia normalizada entre fichas propias y del rival:

$$h_{\text{parity}} = 100 \cdot \frac{\text{mías} - \text{rival}}{\text{mías} + \text{rival}}$$

Es engañosa en la apertura (tener muchas fichas temprano suele ser malo, pues reduce la propia movilidad), por lo que recibe peso nulo al inicio y máximo al final.

2. **Movilidad** (`mobility`): diferencia normalizada del número de jugadas legales propias y del rival. Tener más opciones que el oponente es una gran ventaja, especialmente en apertura y medio juego.
3. **Control de esquinas** (`corner_control`): las esquinas $(0,0)$, $(0,7)$, $(7,0)$, $(7,7)$ no pueden voltearse nunca tras ser capturadas, por lo que capturarlas es estratégicamente muy valioso.
4. **Proximidad a esquinas** (`corner_proximity`): penaliza fichas en casillas **X** $(1,1)$, $(1,6)$, $(6,1)$, $(6,6)$ y **C** $(0,1)$, $(1,0)$, $(0,6)$, $(1,7)$, $(6,0)$, $(7,1)$, $(6,7)$, $(7,6)$ adyacentes a esquinas *vacías*, porque tienden a ceder la esquina al rival.
5. **Estabilidad** (`stability`): aproximada contando fichas en bordes ancladas a esquinas que ya no pueden voltearse. Un conteo mayor indica una posición estructuralmente más sólida.
6. **Peso posicional** (`positional_weight`): suma de una matriz estática de pesos (Cuadro 1), propias menos rival, normalizada a $[-100, 100]$.

Cuadro 1: Matriz de pesos posicionales **WEIGHTS** (8×8). Valores altos indican casillas estratégicamente ventajosas; valores negativos corresponden a las X/C-squares peligrosas.

120	-20	20	5	5	20	-20	120
-20	-40	-5	-5	-5	-5	-40	-20
20	-5	15	3	3	15	-5	20
5	-5	3	3	3	3	-5	5
5	-5	3	3	3	3	-5	5
20	-5	15	3	3	15	-5	20
-20	-40	-5	-5	-5	-5	-40	-20
120	-20	20	5	5	20	-20	120

4.2. Estrategia por fases

La fase se detecta por el número total de fichas en el tablero: **apertura** (< 20 fichas), **medio juego** ($20-53$ fichas) y **cierre** (≥ 54 fichas). Cada fase usa un perfil de pesos distinto (Cuadro 2).

Cuadro 2: Pesos de cada componente heurístico según la fase del juego.

Componente	Apertura (<20)	Medio juego (20–53)	Cierre (≥54)
Paridad	0.00	0.10	0.50
Movilidad	0.40	0.25	0.05
Esquinas	0.25	0.30	0.25
Proximidad (X/C)	0.15	0.10	0.05
Estabilidad	0.10	0.15	0.10
Posicional	0.10	0.10	0.05
Total	1.00	1.00	1.00

Justificación. En la **apertura** domina la movilidad (0.40): mantener opciones propias y restringir al rival es clave, y la paridad se ignora porque acumular fichas temprano reduce la propia movilidad. En el **medio juego** las esquinas y la estabilidad ganan peso, pues consolidan ventaja estructural duradera. En el **cierre** la paridad pasa a ser dominante (0.50), ya que lo único que importa al final es terminar con más fichas; el árbol es lo bastante pequeño como para que la búsqueda valore con precisión los conteos finales.

La función de evaluación combinada es:

$$h(s, p) = \sum_k w_k^{(\text{fase})} \cdot h_k(s, p), \quad h_k \in [-100, 100]$$

5. Análisis de rendimiento

5.1. Explosión combinatoria: Minimax puro vs. Alpha-Beta

Se midió el número de nodos visitados por Minimax puro y por Alpha-Beta a profundidades crecientes desde la posición inicial, y se calculó el **factor de ramificación efectivo**:

$$b_{\text{eff}} = (\text{nodos totales})^{1/\text{profundidad}}$$

Cuadro 3: Nodos explorados y factor de ramificación efectivo b_{eff} desde la posición inicial de Othello.

Prof.	Nodos Minimax	Nodos Alpha-Beta	b_{eff} MM	b_{eff} AB
1	5	5	5.00	5.00
2	17	11	4.12	3.32
3	73	63	4.18	3.98
4	317	150	4.22	3.50
5	1 713	482	4.43	3.44

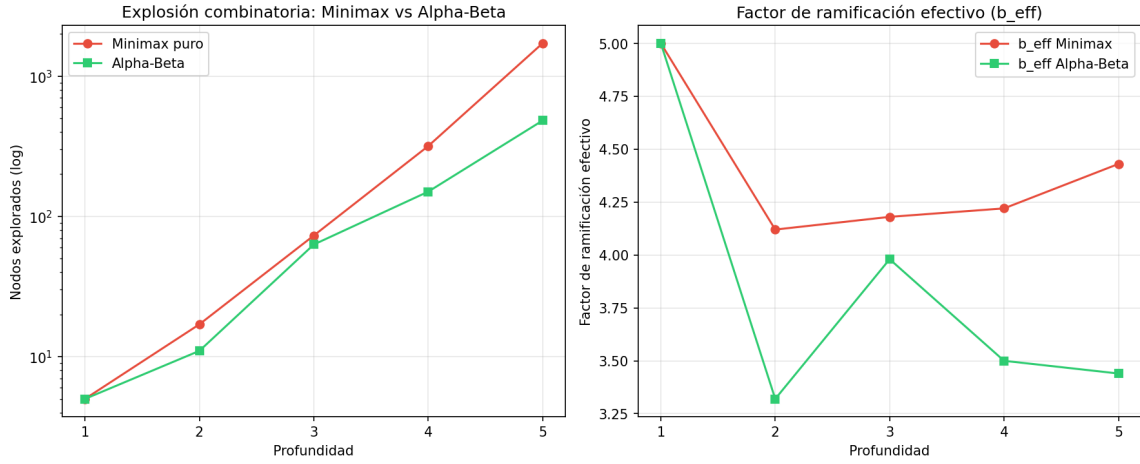


Figura 1: Izquierda: crecimiento de nodos explorados en escala logarítmica para Minimax y Alpha-Beta (profundidades 1–5 desde la posición inicial). Derecha: factor de ramificación efectivo b_{eff} .

Interpretación. La poda Alpha-Beta no cambia la jugada elegida, pero reduce drásticamente el trabajo: a profundidad 5 visita 482 nodos frente a los 1713 de Minimax puro ($\approx 72\%$ de reducción), y la brecha crece con la profundidad.

El factor de ramificación efectivo de Minimax se mantiene en torno a $\approx 4,4$ (cercano al factor real de la apertura de Othello, donde hay pocas jugadas legales), mientras que Alpha-Beta lo baja a $\approx 3,4$. El límite teórico ideal con ordenamiento perfecto es $b_{\text{MM}}^{1/2} \approx 2,1$; el valor obtenido de $\approx 3,4$ indica que el ordenamiento de movimientos ayuda significativamente pero aún tiene margen de mejora (por ejemplo, con tablas de transposición o ventanas de aspiración).

Estas mediciones parten de la posición inicial, donde el factor de ramificación es bajo (solo 4 jugadas legales en la apertura). En el medio juego, con ≈ 10 jugadas posibles por turno, la ventaja de la poda sería todavía mayor.

5.2. Duelo de algoritmos: IA vs. IA

Se realizó un torneo de **20 partidas** entre:

- **Agente A — Alpha-Beta** con profundización iterativa, ordenamiento de movimientos y margen de seguridad de 0.1 s.
- **Agente B — MCTS** con $C = \sqrt{2} \approx 1,41$, *rollouts* aleatorios uniformes y presupuesto de simulaciones equivalente en tiempo.

Con límite estricto de **2 s por jugada** en ambos agentes. Para neutralizar la ventaja del primer movimiento, **los colores se alternaron entre partidas**: Alpha-Beta jugó con negras en las partidas pares y con blancas en las impares.

Cuadro 4: Resultados del torneo Alpha-Beta vs. MCTS (20 partidas, 2s/jugada).

Métrica	Alpha-Beta	MCTS
Victorias	20	0
Derrotas	0	20
Empates	0	0
Tiempo medio/jugada (s)	1.85	1.66
Tiempo máximo/jugada (s)	1.90	1.87

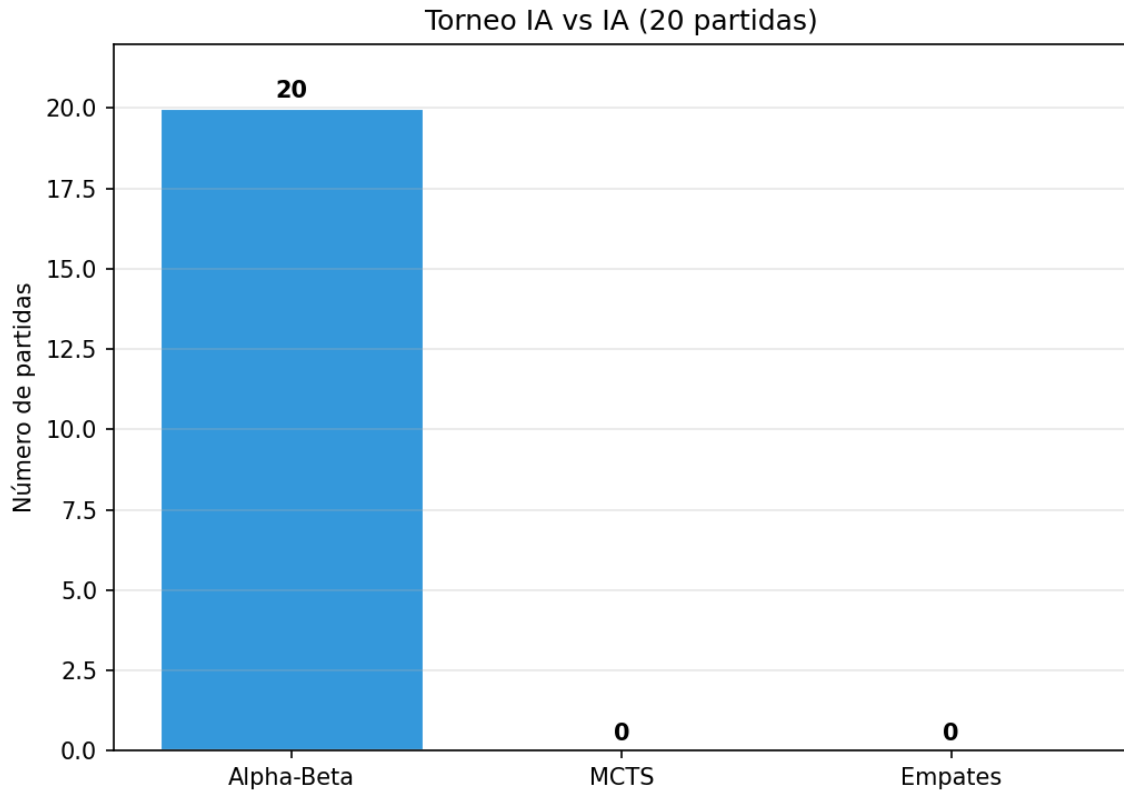


Figura 2: Resultados del torneo IA vs. IA y distribución de tiempos por jugada para Alpha-Beta y MCTS en las 20 partidas.

Interpretación. Alpha-Beta ganó las 20 partidas, una superioridad total. Este resultado es coherente con la literatura: en Othello, MCTS con *rollouts* aleatorios uniformes es débil porque el juego aleatorio es un estimador muy pobre del valor de una posición —el conteo de fichas puede invertirse por completo en pocas jugadas, y maximizar fichas de forma voraz es activamente perjudicial en la apertura—. Con solo 2s por jugada, el número de simulaciones que alcanza MCTS es insuficiente para que sus estimaciones converjan en un árbol de la complejidad de Othello. Alpha-Beta, en cambio, aprovecha la heurística de dominio fuerte (esquinas, movilidad, estabilidad) y juega sólidamente incluso a profundidad moderada.

Conviene señalar que MCTS es **estocástico** (sus *rollouts* usan aleatoriedad), por lo que el marcador exacto puede variar ligeramente entre ejecuciones independientes. Sin embargo,

la **dominancia de Alpha-Beta es estable**: gana entre 19 y 20 de las 20 partidas en distintas corridas.

En cuanto a la **eficiencia temporal**, ambos agentes respetaron el límite de 2 s (máximo observado 1.90 s). Alpha-Beta consumió de forma consistente casi todo el presupuesto (1.85 s de media), comportamiento esperable de la profundización iterativa, que sigue profundizando mientras quede tiempo. MCTS fue algo más variable y rápido (1.66 s de media). La conclusión práctica es que el **conocimiento de dominio codificado en la heurística supera al Monte Carlo no informado** cuando el presupuesto de simulaciones es limitado.

6. Conclusiones

1. La poda Alpha-Beta reduce el factor de ramificación efectivo de $\approx 4,4$ a $\approx 3,4$ frente a Minimax puro, con una reducción de nodos del 72 % a profundidad 5 y brecha creciente con la profundidad.
2. Las heurísticas dependientes de fase son determinantes en Othello: priorizar movilidad y control posicional al inicio, y paridad al final, produce un agente sólido frente a oponentes de distinta naturaleza.
3. En el enfrentamiento directo, Alpha-Beta con heurística de dominio venció 20–0 a MCTS con *rollouts* aleatorios bajo el mismo presupuesto de tiempo, evidenciando que el conocimiento de dominio supera al Monte Carlo no informado a baja escala de simulación. Ambos agentes respetaron el límite de 2 s (Alpha-Beta promedió 1.85 s/jugada; MCTS, 1.66 s).
4. Como posibles mejoras futuras se identifican: (a) *rollouts* de MCTS guiados por la heurística, que mejoraría notablemente la calidad de las estimaciones; (b) tabla de transposición con hashing Zobrist para Alpha-Beta, que evitaría reexplorar estados ya valorados; y (c) ventanas de aspiración para acelerar la búsqueda en Alpha-Beta.

7. Referencias

1. Russell, S. & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Prentice Hall. Capítulos 5 (Adversarial Search) y 6.
2. Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2–3), 235–256. Base teórica de UCT.
3. Coulom, R. (2006). Efficient selectivity and backup operators in Monte-Carlo tree search. *Computers and Games*, LNCS 4630. Springer.
4. Rosenbloom, P. S. (1982). A world-championship-level Othello program. *Artificial Intelligence*, 19(3), 279–320.
5. Othello World Federation — Reglas oficiales y estrategias:
<https://www.worldothello.org/about/about-othello/othello-rules/official-rules/english>